AMAZON SDE-1

DSA INTERVIEW ROADMAP

A comprehensive pattern guide, curated problem list, and five-day execution plan

CORE TARGET	PATTERNS	SPRINT
60 problems	**17 families**	**5 focused days**

Designed for pattern recognition, explanation, dry runs, clean C++ coding, and complexity analysis.

Use this as a sprint checklist - not as a requirement to solve every stretch problem.

How to Use This PDF

Pass 1 (learning): spend 20-30 minutes per core problem. Explain brute force, identify the pattern, derive the optimized approach, code, and dry-run.

Pass 2 (recall): revisit the same problem without looking at code. Write only the invariant, state, or key observation, then implement in 15-20 minutes.

Use the stretch problems only after the day's core set is complete. Hard problems are included to deepen patterns, not to dominate the sprint.

For every solution, say the time and space complexity aloud and name the edge cases before submitting.

Preferred practice language: modern C++17/C++20 using vector, string, unordered_map, unordered_set, queue, stack, priority_queue, and algorithms.

Scope note: Amazon's official software-development interview guidance explicitly emphasizes data structures, algorithms, coding, programming-language fluency, and complexity-aware problem solving. This roadmap prioritizes those areas; it is not a list of leaked or guaranteed questions.

Five-Day Execution Plan

Day 1: Arrays, hashing, two pointers, windows, intervals and binary-search fundamentals

Daily cadence: 30 min pattern review -> 3 x 90 min problem blocks -> 45 min recall/re-code -> 15 min error log.

[ ] Problem	Primary pattern
[ ] Two Sum	Arrays and Hashing
[ ] Group Anagrams	Arrays and Hashing
[ ] Top K Frequent Elements	Arrays and Hashing
[ ] Product of Array Except Self	Arrays and Hashing
[ ] Longest Consecutive Sequence	Arrays and Hashing
[ ] 3Sum	Two Pointers
[ ] Container With Most Water	Two Pointers
[ ] Longest Substring Without Repeating Characters	Sliding Window
[ ] Longest Repeating Character Replacement	Sliding Window
[ ] Permutation in String	Sliding Window
[ ] Merge Intervals	Prefix Sums and Intervals
[ ] Binary Search	Binary Search

End-of-day gate: re-code any three problems from memory and explain one solution in interviewer style without notes.

Day 2: Stacks, linked lists, heaps and binary-search-on-answer reinforcement

Daily cadence: 30 min pattern review -> 3 x 90 min problem blocks -> 45 min recall/re-code -> 15 min error log.

[ ] Problem	Primary pattern
[ ] Search in Rotated Sorted Array	Binary Search
[ ] Koko Eating Bananas	Binary Search
[ ] Valid Parentheses	Stack, Queue and Monotonic Stack
[ ] Daily Temperatures	Stack, Queue and Monotonic Stack
[ ] Decode String	Stack, Queue and Monotonic Stack
[ ] Reverse Linked List	Linked Lists and Pointer Manipulation
[ ] Linked List Cycle	Linked Lists and Pointer Manipulation
[ ] Remove Nth Node From End	Linked Lists and Pointer Manipulation
[ ] Reorder List	Linked Lists and Pointer Manipulation
[ ] Kth Largest Element in an Array	Heaps and Top-K

[ ] Problem	Primary pattern
[ ] K Closest Points to Origin	Heaps and Top-K
[ ] Task Scheduler	Heaps and Top-K

End-of-day gate: re-code any three problems from memory and explain one solution in interviewer style without notes.

Day 3: Trees first, then grid/graph traversal

Daily cadence: 30 min pattern review -> 3 x 90 min problem blocks -> 45 min recall/re-code -> 15 min error log.

[ ] Problem	Primary pattern
[ ] Maximum Depth of Binary Tree	Binary Trees and BSTs
[ ] Binary Tree Level Order Traversal	Binary Trees and BSTs
[ ] Diameter of Binary Tree	Binary Trees and BSTs
[ ] Balanced Binary Tree	Binary Trees and BSTs
[ ] Lowest Common Ancestor of a Binary Tree	Binary Trees and BSTs
[ ] Validate Binary Search Tree	Binary Trees and BSTs
[ ] Kth Smallest Element in a BST	Binary Trees and BSTs
[ ] Construct Tree from Preorder and Inorder	Binary Trees and BSTs
[ ] Binary Tree Right Side View	Binary Trees and BSTs
[ ] Number of Islands	Graph Traversal and Grid BFS/DFS
[ ] Rotting Oranges	Graph Traversal and Grid BFS/DFS
[ ] Clone Graph	Graph Traversal and Grid BFS/DFS

End-of-day gate: re-code any three problems from memory and explain one solution in interviewer style without notes.

Day 4: Graph dependencies, union-find, shortest paths, backtracking and trie

Daily cadence: 30 min pattern review -> 3 x 90 min problem blocks -> 45 min recall/re-code -> 15 min error log.

[ ] Problem	Primary pattern
[ ] Course Schedule	Topological Sort and Union-Find
[ ] Course Schedule II	Topological Sort and Union-Find
[ ] Redundant Connection	Topological Sort and Union-Find
[ ] Accounts Merge	Topological Sort and Union-Find
[ ] Network Delay Time	Shortest Paths
[ ] Cheapest Flights Within K Stops	Shortest Paths
[ ] Subsets	Backtracking
[ ] Permutations	Backtracking
[ ] Combination Sum	Backtracking
[ ] Generate Parentheses	Backtracking
[ ] Word Search	Backtracking
[ ] Implement Trie (Prefix Tree)	Tries, Bit Manipulation and Matrices

End-of-day gate: re-code any three problems from memory and explain one solution in interviewer style without notes.

Day 5: Greedy, dynamic programming, design problem, and two timed mocks

Daily cadence: 30 min pattern review -> 3 x 90 min problem blocks -> 45 min recall/re-code -> 15 min error log.

[ ] Problem	Primary pattern
[ ] Maximum Subarray	Greedy
[ ] Jump Game	Greedy
[ ] Climbing Stairs	Dynamic Programming

[ ] Problem	Primary pattern
[ ] House Robber	Dynamic Programming
[ ] Coin Change	Dynamic Programming
[ ] Longest Increasing Subsequence	Dynamic Programming
[ ] Longest Common Subsequence	Dynamic Programming
[ ] Word Break	Dynamic Programming
[ ] Unique Paths	Dynamic Programming
[ ] Decode Ways	Dynamic Programming
[ ] Partition Equal Subset Sum	Dynamic Programming
[ ] Time Based Key-Value Store	Design-Oriented Data Structure Problems

End-of-day gate: re-code any three problems from memory and explain one solution in interviewer style without notes.

Pattern Recognition Playbook

1. Arrays and Hashing

Recognition cue: Fast lookup, counting, deduplication, grouping, complements, or subarray frequency.

Implementation invariant: Use unordered_map/unordered_set; define exactly what the key represents; update in the correct order.

2. Two Pointers

Recognition cue: Sorted input, pair/triplet constraints, in-place compaction, palindrome checks, or opposite-end movement.

Implementation invariant: State why moving left or right cannot discard a valid better answer; sort first only when index identity is not required.

3. Sliding Window

Recognition cue: Longest/shortest contiguous segment with a validity condition or bounded budget.

Implementation invariant: Expand right, update state, shrink while invalid, then record the answer at the correct moment.

4. Prefix Sums and Intervals

Recognition cue: Many range queries, subarray totals, overlapping time ranges, or scheduling conflicts.

Implementation invariant: For intervals, sort by start time and define whether touching endpoints overlap; for prefix sums, define prefix[0] = 0.

5. Binary Search

Recognition cue: Sorted data, rotated order, boundary finding, or a monotonic yes/no feasibility function.

Implementation invariant: Write the invariant before coding; decide whether the answer space is indices or values; test one-element and no-solution cases.

6. Stack, Queue and Monotonic Stack

Recognition cue: Nested structure, undo/order reversal, nearest greater/smaller element, expression evaluation, or FIFO simulation.

Implementation invariant: A monotonic stack stores unresolved candidates; say what monotonic property it maintains and when an item becomes resolved.

7. Linked Lists and Pointer Manipulation

Recognition cue: In-place reversal, cycle detection, middle finding, merging, or node reordering.

Implementation invariant: Draw pointers before changing them; use a dummy node for head edge cases; save next before rewiring current.

8. Binary Trees and BSTs

Recognition cue: Hierarchical recursion, subtree aggregation, level-by-level traversal, ancestor queries, or ordered-tree constraints.

Implementation invariant: Define precisely what dfs(node) returns; choose preorder/inorder/postorder based on when the parent needs child results.

9. Heaps and Top-K

Recognition cue: Repeatedly need the smallest/largest candidate, top k elements, streaming order statistics, or merging sorted sources.

Implementation invariant: Choose min-heap vs max-heap by asking which element should be evicted or selected next; state heap size and total complexity.

10. Graph Traversal and Grid BFS/DFS

Recognition cue: Connectivity, components, shortest unweighted path, flood fill, dependencies, or state propagation.

Implementation invariant: Define node/state, neighbors, visited timing, and whether traversal is BFS or DFS; mark visited when enqueued to avoid duplicates.

11. Topological Sort and Union-Find

Recognition cue: Prerequisites, dependency order, cycle detection in directed graphs, or dynamic connectivity in undirected graphs.

Implementation invariant: Topological sort: indegree + queue or DFS colors. Union-find: path compression + union by rank/size.

12. Shortest Paths

Recognition cue: Minimum cost/time in a weighted graph; edge weights determine BFS, 0-1 BFS, Dijkstra, or Bellman-Ford.

Implementation invariant: Dijkstra requires non-negative weights; skip stale heap entries; explain distance relaxation explicitly.

13. Backtracking

Recognition cue: Enumerate combinations/permutations, make reversible choices, or search a constrained decision tree.

Implementation invariant: Choose -> recurse -> unchoose. State the candidate set, duplicate policy, pruning rule, and base case.

14. Greedy

Recognition cue: A locally optimal choice can be proven safe through exchange, earliest finish, farthest reach, or invariant reasoning.

Implementation invariant: Do not label a solution greedy without a proof sketch; state why an alternative choice cannot produce a better future.

15. Dynamic Programming

Recognition cue: Overlapping subproblems, choices across positions, optimization/counting, or a brute-force recursion with repeated states.

Implementation invariant: Define state in one sentence, transition, base cases, evaluation order, and final state. Start memoized, then compress if useful.

16. Tries, Bit Manipulation and Matrices

Recognition cue: Prefix lookup, character branching, XOR identities, fixed-width binary state, or in-place matrix transformation.

Implementation invariant: Trie nodes represent prefixes; bit solutions need an explicit identity; matrix solutions need boundary/order checks.

17. Design-Oriented Data Structure Problems

Recognition cue: The problem asks for a class/API supporting strict time bounds across multiple operations.

Implementation invariant: Translate each operation into required complexity, then combine data structures so one supplies ordering and another lookup.

Comprehensive Pattern-Based Problem List

Legend: C = core for the five-day sprint. S = stretch after core completion. Click a problem title to open it.

1. Arrays and Hashing

Look for: Fast lookup, counting, deduplication, grouping, complements, or subarray frequency.

P	Problem	What it trains	Level
C	Two Sum	Find two indices whose values add to a target; trains complement lookup.	Easy
S	Contains Duplicate	Detect repeated values; set membership and early exit.	Easy
S	Valid Anagram	Compare character frequencies without sorting when possible.	Easy
C	Group Anagrams	Build a canonical key for equivalent strings.	Medium
C	Top K Frequent Elements	Frequency map plus buckets or a heap.	Medium
C	Product of Array Except Self	Prefix/suffix products without division.	Medium
C	Longest Consecutive Sequence	Start only at sequence heads; expected O(n).	Medium
S	Subarray Sum Equals K	Prefix sum frequencies for signed values.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

2. Two Pointers

Look for: Sorted input, pair/triplet constraints, in-place compaction, palindrome checks, or opposite-end movement.

P	Problem	What it trains	Level
S	Valid Palindrome	Skip non-alphanumeric characters and compare inward.	Easy
S	Two Sum II - Sorted Input	Use monotonic movement on a sorted array.	Medium
C	3Sum	Sort, fix one value, then solve a duplicate-safe two-sum.	Medium
C	Container With Most Water	Move the limiting wall while tracking the best area.	Medium
S	Move Zeroes	Stable in-place compaction with a write pointer.	Easy
S	Trapping Rain Water	Two-sided maxima and a provably safe pointer move.	Hard

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

3. Sliding Window

Look for: Longest/shortest contiguous segment with a validity condition or bounded budget.

P	Problem	What it trains	Level
S	Best Time to Buy and Sell Stock	Track the minimum prefix price and best profit.	Easy
C	Longest Substring Without Repeating Characters	Window with last-seen positions or counts.	Medium
C	Longest Repeating Character Replacement	Maintain max frequency and replacement budget.	Medium
C	Permutation in String	Fixed-size frequency window for an anagram match.	Medium
S	Max Consecutive Ones III	Longest window containing at most k zeroes.	Medium

P	Problem	What it trains	Level
S	Minimum Window Substring	Smallest valid covering window with required counts.	Hard

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

4. Prefix Sums and Intervals

Look for: Many range queries, subarray totals, overlapping time ranges, or scheduling conflicts.

P	Problem	What it trains	Level
S	Range Sum Query - Immutable	Build prefix sums for O(1) range queries.	Easy
C	Merge Intervals	Sort and merge overlapping ranges.	Medium
S	Insert Interval	Insert and merge in one pass through sorted ranges.	Medium
S	Non-overlapping Intervals	Greedy removal by earliest finishing endpoint.	Medium
S	Interval List Intersections	Two pointers across two sorted interval lists.	Medium
S	Car Pooling	Difference array / event sweep over capacity changes.	Medium
S	Continuous Subarray Sum	Prefix remainders and earliest index handling.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

5. Binary Search

Look for: Sorted data, rotated order, boundary finding, or a monotonic yes/no feasibility function.

P	Problem	What it trains	Level
C	Binary Search	Exact lookup with correct boundaries.	Easy
S	Search Insert Position	Lower-bound style first valid position.	Easy
C	Search in Rotated Sorted Array	Identify the sorted half before discarding it.	Medium
S	Find Minimum in Rotated Sorted Array	Compare mid with the right boundary.	Medium
C	Koko Eating Bananas	Binary search the minimum feasible rate.	Medium
S	Capacity to Ship Packages Within D Days	Binary search capacity using a greedy feasibility check.	Medium
S	Median of Two Sorted Arrays	Partition two arrays to balance left and right halves.	Hard

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

6. Stack, Queue and Monotonic Stack

Look for: Nested structure, undo/order reversal, nearest greater/smaller element, expression evaluation, or FIFO simulation.

P	Problem	What it trains	Level
C	Valid Parentheses	Match nested delimiters with a stack.	Easy
S	Min Stack	Support O(1) minimum queries alongside push/pop.	Medium
C	Daily Temperatures	Resolve next-warmer-day indices using a decreasing stack.	Medium
S	Evaluate Reverse Polish Notation	Evaluate postfix expressions safely.	Medium
C	Decode String	Parse nested repetition with stacks or recursion.	Medium

P	Problem	What it trains	Level
S	Largest Rectangle in Histogram	Nearest-smaller boundaries via a monotonic stack.	Hard
S	Implement Queue using Stacks	Amortized analysis with lazy transfer.	Easy

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

7. Linked Lists and Pointer Manipulation

Look for: In-place reversal, cycle detection, middle finding, merging, or node reordering.

P	Problem	What it trains	Level
C	Reverse Linked List	Iterative pointer reversal and invariant.	Easy
S	Merge Two Sorted Lists	Dummy-head merge of two ordered lists.	Easy
C	Linked List Cycle	Floyd slow/fast pointer detection.	Easy
C	Remove Nth Node From End	Fixed gap between fast and slow pointers.	Medium
C	Reorder List	Find middle, reverse second half, then weave.	Medium
S	Copy List with Random Pointer	Clone mapping or interleaving technique.	Medium
S	Add Two Numbers	Digit-wise addition with carry and unequal lengths.	Medium
S	LRU Cache	Hash map plus doubly linked list for O(1) operations.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

8. Binary Trees and BSTs

Look for: Hierarchical recursion, subtree aggregation, level-by-level traversal, ancestor queries, or ordered-tree constraints.

P	Problem	What it trains	Level
C	Maximum Depth of Binary Tree	Base case plus recursive height.	Easy
S	Same Tree	Structural and value equality.	Easy
S	Invert Binary Tree	Recursive or BFS child swapping.	Easy
C	Binary Tree Level Order Traversal	Queue-based BFS by levels.	Medium
C	Diameter of Binary Tree	Postorder height with a global path answer.	Easy
C	Balanced Binary Tree	Return height or a failure sentinel in one pass.	Easy
C	Lowest Common Ancestor of a Binary Tree	Combine left/right recursive evidence.	Medium
C	Validate Binary Search Tree	Propagate strict lower and upper bounds.	Medium
C	Kth Smallest Element in a BST	Inorder traversal produces sorted order.	Medium
C	Construct Tree from Preorder and Inorder	Root selection plus indexed inorder partitions.	Medium
C	Binary Tree Right Side View	Level-order last node or right-first DFS.	Medium
S	Serialize and Deserialize Binary Tree	Design an unambiguous traversal encoding.	Hard
S	Binary Tree Maximum Path Sum	Postorder gain; ignore negative child contributions.	Hard

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

9. Heaps and Top-K

Look for: Repeatedly need the smallest/largest candidate, top k elements, streaming order statistics, or merging sorted sources.

P	Problem	What it trains	Level
C	Kth Largest Element in an Array	Size-k min-heap or quickselect.	Medium
C	K Closest Points to Origin	Top-k selection by distance.	Medium
C	Task Scheduler	Greedy frequency scheduling with idle-slot reasoning or heap simulation.	Medium
S	Merge K Sorted Lists	Heap of current list heads.	Hard
S	Find Median from Data Stream	Balance a max-heap and min-heap.	Hard
S	Reorganize String	Always place the most frequent currently valid character.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

10. Graph Traversal and Grid BFS/DFS

Look for: Connectivity, components, shortest unweighted path, flood fill, dependencies, or state propagation.

P	Problem	What it trains	Level
C	Number of Islands	Connected components in a grid.	Medium
C	Clone Graph	Graph copy with old-to-new mapping.	Medium
C	Rotting Oranges	Multi-source BFS measured in layers.	Medium
S	Pacific Atlantic Water Flow	Reverse reachability from both boundaries.	Medium
S	Number of Provinces	Count connected components in an adjacency matrix.	Medium
S	Word Ladder	Shortest transformation path with BFS.	Hard
S	Minimum Height Trees	Repeatedly trim leaves to find graph centers.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

11. Topological Sort and Union-Find

Look for: Prerequisites, dependency order, cycle detection in directed graphs, or dynamic connectivity in undirected graphs.

P	Problem	What it trains	Level
C	Course Schedule	Detect whether a prerequisite graph has a cycle.	Medium
C	Course Schedule II	Return one valid topological ordering.	Medium
C	Redundant Connection	Find the first edge that closes a cycle using DSU.	Medium
C	Accounts Merge	Union records sharing identifiers.	Medium
S	Evaluate Division	Weighted graph traversal for ratio queries.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

12. Shortest Paths

Look for: Minimum cost/time in a weighted graph; edge weights determine BFS, 0-1 BFS, Dijkstra, or Bellman-Ford.

P	Problem	What it trains	Level
C	Network Delay Time	Dijkstra from one source to all nodes.	Medium

P	Problem	What it trains	Level
C	Cheapest Flights Within K Stops	Shortest path with a stop-count state constraint.	Medium
S	Path With Minimum Effort	Minimize the maximum edge along a path.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

13. Backtracking

Look for: Enumerate combinations/permutations, make reversible choices, or search a constrained decision tree.

P	Problem	What it trains	Level
C	Subsets	Include/exclude or growing-path enumeration.	Medium
C	Permutations	Track used choices or swap in place.	Medium
C	Combination Sum	Reuse candidates while pruning by remaining target.	Medium
C	Generate Parentheses	Build only valid prefixes using open/close counts.	Medium
C	Word Search	Grid backtracking with temporary visited state.	Medium
S	Palindrome Partitioning	Precompute/check palindromes while partitioning.	Medium
S	N-Queens	Column and diagonal constraints with pruning.	Hard

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

14. Greedy

Look for: A locally optimal choice can be proven safe through exchange, earliest finish, farthest reach, or invariant reasoning.

P	Problem	What it trains	Level
C	Maximum Subarray	Kadane: best subarray ending at each index.	Medium
C	Jump Game	Track the farthest reachable index.	Medium
S	Gas Station	Total feasibility plus reset after a failing prefix.	Medium
S	Partition Labels	Close a segment at the farthest last occurrence.	Medium
S	Hand of Straights	Greedily start groups from the smallest remaining card.	Medium
S	Merge Triplets to Form Target Triplet	Ignore unusable triplets and cover each target coordinate.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

15. Dynamic Programming

Look for: Overlapping subproblems, choices across positions, optimization/counting, or a brute-force recursion with repeated states.

P	Problem	What it trains	Level
C	Climbing Stairs	One-dimensional counting DP.	Easy
C	House Robber	Choose rob/skip with adjacent exclusion.	Medium
S	House Robber II	Reduce a cycle to two linear cases.	Medium
C	Coin Change	Unbounded minimum-count DP.	Medium
C	Longest Increasing Subsequence	O(n^2) DP, then understand O(n log n) tails.	Medium

P	Problem	What it trains	Level
C	Longest Common Subsequence	Two-string 2D DP.	Medium
C	Word Break	Prefix segmentation with dictionary membership.	Medium
C	Unique Paths	Grid path-counting DP.	Medium
C	Decode Ways	Count valid one- and two-character decodings.	Medium
C	Partition Equal Subset Sum	0/1 knapsack reachability.	Medium
S	Best Time to Buy/Sell Stock with Cooldown	State-machine DP for hold/sold/rest.	Medium
S	Target Sum	Transform signs into subset-count DP.	Medium
S	Edit Distance	Insert/delete/replace transition over two prefixes.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

16. Tries, Bit Manipulation and Matrices

Look for: Prefix lookup, character branching, XOR identities, fixed-width binary state, or in-place matrix transformation.

P	Problem	What it trains	Level
C	Implement Trie (Prefix Tree)	Insert, exact search, and prefix search.	Medium
S	Design Add and Search Words	Trie plus wildcard DFS.	Medium
S	Word Search II	Trie-guided grid search with pruning.	Hard
S	Single Number	Use XOR cancellation.	Easy
S	Counting Bits	Derive answers from smaller states.	Easy
S	Missing Number	XOR or arithmetic invariant.	Easy
S	Sum of Two Integers	Bitwise addition with carry.	Medium
S	Set Matrix Zeroes	Use first row/column as in-place markers.	Medium
S	Spiral Matrix	Maintain shrinking boundaries.	Medium
S	Rotate Image	Transpose then reverse, in place.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

17. Design-Oriented Data Structure Problems

Look for: The problem asks for a class/API supporting strict time bounds across multiple operations.

P	Problem	What it trains	Level
C	Time Based Key-Value Store	Map each key to time-sorted values; binary search by timestamp.	Medium
S	Insert Delete GetRandom O(1)	Vector plus value-to-index map and swap-delete.	Medium
S	Design Twitter	Timestamped feeds plus heap merge of recent posts.	Medium

Interview drill: identify the trigger, state the invariant/state, give complexity, and name one failure mode.

The 35-Minute Interview Workflow

0-3 min - Clarify

Restate input/output. Ask about constraints, duplicates, null/empty input, value ranges, mutability, and whether indices or values are required.

3-7 min - Baseline

Give a simple brute-force method and its complexity. This proves understanding and creates a reference point for optimization.

7-12 min - Pattern and proof
Name the pattern only after explaining the key observation. State the invariant, DP state, heap meaning, or traversal state.

12-25 min - Code
Write compilable code with descriptive names, helper functions where useful, and no unnecessary framework. Narrate critical lines, not every keystroke.

25-30 min - Dry run
Use a non-trivial example. Track pointers, window counts, queue/stack content, recursion returns, or DP cells.

30-35 min - Test and analyze
Cover empty/singleton/duplicate/extreme cases. State time and auxiliary space, including recursion stack and output storage conventions.

C++ Interview Checklist

Use long long when sums/products can exceed 32-bit int.
For priority_queue: max-heap by default; min-heap uses priority_queue<T, vector<T>, greater<T>>.
Avoid accidental O(n) string/vector copies in recursive calls; pass by reference when safe.
Use const references for read-only inputs and reserve hash maps/sets only when it materially helps.
Remember unordered_map worst case is theoretically O(n), though expected operations are O(1).
For BFS, mark visited when enqueuing, not when dequeuing.
For binary search, avoid overflow: mid = left + (right-left)/2.
For trees/graphs, account for recursion depth; mention iterative alternatives if depth may be large.
Do not use non-standard variable-length arrays; use vector.
Compile mentally for missing semicolons, incorrect comparators, iterator invalidation, and signed/unsigned mismatches.

Two Timed Mock Sets for Day 5

Mock A - 70 minutes

[ ] Longest Substring Without Repeating Characters - Window + hash; 20 minutes coding + 3 minutes test/complexity.
[ ] Lowest Common Ancestor of a Binary Tree - Tree recursion; 20 minutes coding + 3 minutes test/complexity.
[ ] Coin Change - DP; 20 minutes coding + 3 minutes test/complexity.

Mock rule: no solution videos or editorial until the full set is finished. Record the first moment you became stuck.

Mock B - 70 minutes

[ ] Merge Intervals - Sorting + intervals; 20 minutes coding + 3 minutes test/complexity.
[ ] Course Schedule - Topological sort; 20 minutes coding + 3 minutes test/complexity.
[ ] Kth Largest Element in an Array - Heap / selection; 20 minutes coding + 3 minutes test/complexity.

Mock rule: no solution videos or editorial until the full set is finished. Record the first moment you became stuck.

Error Log Template

Problem	Missed trigger	Bug / gap	Correct invariant	Re-code date

Complexity Quick Reference

Operation	Typical time	Reminder
Array access	O(1)	Direct index
Hash map/set lookup	Expected O(1)	Worst-case O(n)
Balanced ordered map/set	O(log n)	Keeps keys sorted
Sorting	O(n log n)	Comparison sorting
Binary search	O(log n)	Monotonic/sorted search space
Heap push/pop	O(log n)	Top is O(1)
BFS/DFS graph	O(V+E)	Adjacency list
Tree traversal	O(n)	Each node once
Union-find	Near O(1)	Amortized with compression/rank
Dijkstra with heap	O((V+E) log V)	Non-negative weights
Backtracking	Usually exponential	Depends on branching/pruning
2D DP	Often O(nm)	State dimensions determine cost

Final 24-Hour Checklist

[ ] Re-code Two Sum, sliding window, binary search, reverse linked list, tree DFS/BFS, Number of Islands, Course Schedule, heap top-k, and one DP from memory.
[ ] Review STL syntax, comparator syntax, integer overflow, recursion base cases, and graph visited handling.
[ ] Perform one 45-minute mock with verbal explanation and no autocomplete dependence.
[ ] Review the error log only; do not start a large new topic.
[ ] Prepare a 60-second explanation of your problem-solving method and how you validate correctness.
[ ] Sleep adequately and keep the final warm-up to two easy problems plus one medium pattern you already know.

Official and Practice Resources

- **Amazon - Software development interview topics**
- **Amazon - Software development interview preparation**
- **Amazon - SDE online assessment and interview prep**
- **LeetCode - Algorithms problem set**
- **HackerRank - Interview Preparation Kit**

Total curated problems: 60 core + 62 stretch = 122.

Problem links and availability may change over time. This guide intentionally avoids claiming that any item is guaranteed to appear in a specific Amazon interview.